LOW LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant using LangGraph & HITL

Further More Information:

- P Jeevan
- Advanced Generative AI Internship
- Innomatics Research Labs

TABLE OF CONTENTS

1 INTRODUCTION

The Low-Level Design defines the internal implementation structure of the RAG-based support assistant, including:

- **module responsibilities**
- **data structures**
- **workflow states**
- **routing logic**
- **API contracts**
- **error handling**

2 MODULE-LEVEL DESIGN

2.1 Document Processing Module

Responsibilities

- **load PDFs**
- **extract raw text**
- **preprocess formatting**

Input

PDF file

Output

Cleaned textual content

2.2 Chunking Module

Responsibilities

- **split text into chunks**
- **maintain overlap**

Parameters

- **chunk_size**
- **chunk_overlap**

Output

Semantic text chunks

2.3 Embedding Module

Responsibilities

- **convert chunks into vectors**

Output Format

```
{
  "chunk_id": "c101",
  "embedding": [0.234, 0.991, ...]
}
```

2.4 Vector Storage Module

Responsibilities

- **store embeddings**
- **manage retrieval indexing**

Database

ChromaDB

2.5 Retrieval Module

Responsibilities

- **query embedding generation**
- **top-k retrieval**

Retrieval Strategy

Cosine similarity search

2.6 Query Processing Module

Responsibilities

- **construct prompts**
- **inject retrieved context**
- **invoke LLM**

2.7 Graph Execution Module

Responsibilities

- **execute LangGraph workflow**
- **manage node transitions**
- **maintain state**

2.8 HITL Module

Responsibilities

- **trigger escalation**
- **forward unresolved queries**
- **integrate human response**

3 DATA STRUCTURES

Document Schema

```
{
 "doc_id": "D001",
 "title": "Support Manual",
 "content": "..."
}
```

Chunk Schema

```
{
 "chunk_id": "C101",
 "doc_id": "D001",
 "text": "...",
 "metadata": {
  "page": 2
 }
}
```

Query Schema

```
{
 "query_id": "Q001",
 "user_query": "Reset password issue"
}
```

Response Schema

```
{
 "response": "...",
 "confidence_score": 0.82,
 "escalated": false
}
```

State Object

```
{
 "query": "",
 "retrieved_chunks": [],
 "response": "",
 "confidence": 0.0,
 "escalation_required": false
}
```

4 LANGGRAPH WORKFLOW DETAILS

Nodes

Node	Responsibility
Input Node	Receive query
Retrieval Node	Fetch chunks
Processing Node	Generate answer
Decision Node	Evaluate confidence
Output Node	Return response

Node	Responsibility
Escalation Node	Trigger HITL

5 CONDITIONAL ROUTING LOGIC

Automated Response Conditions

- **sufficient retrieval confidence**
- **relevant context available**
- **low ambiguity**

Escalation Conditions

- **confidence below threshold**
- **no relevant chunks**
- **sensitive customer issue**
- **complex query intent**

Example Logic

```
if confidence < 0.65:
   escalate = True
else:
   escalate = False
```

6 HITL INTERNAL WORKFLOW

Flow

1. **Escalation triggered**
2. **Human agent notified**
3. **Context shared**
4. **Human submits response**
5. **Response integrated**
6. **Final answer returned**

7 API DESIGN

Query API

Endpoint

POST /query

Request

{
"query": "How do I reset my password?"
}

Response

{
"response": "To reset your password...",
"confidence": 0.87,
"escalated": false
}

8 ERROR HANDLING

Missing PDF

- **validation checks**

Empty Retrieval

- **fallback messaging**

LLM Failure

- **retry mechanism**

Embedding Failure

- **logging and exception handling**

9 PERFORMANCE OPTIMIZATION

Mention

- **embedding caching**
- **async retrieval**
- **chunk indexing**
- **retrieval filtering**
- **top-k optimization**

10 **TESTING STRATEGY**

Unit Testing

- **chunking**
- **retrieval**
- **routing**

Integration Testing

- **full workflow execution**

Sample Queries

- **simple support query**

- **ambiguous query**
- **escalation scenario**

11 CONCLUSION

The LLD defines a modular and extensible implementation architecture capable of:

- **semantic retrieval**
- **intelligent routing**
- **workflow orchestration**
- **reliable escalation handling**

The design supports future scalability and production-oriented AI deployment.